Занятие 46.2. Ссылки

До этого момента мы успели рассмотреть 2 основных типа переменных:

- обычные переменные, которые хранят значения напрямую;
- **указатели**, которые хранят адрес другого значения (или **null**), для доступа к которым выполняется операция разыменования указателя.

Ссылки — это третий базовый тип переменных в языке C++.

Оглавление:

1. Ссылки
2. Ссылки в качестве псевдонимов
3. Краткий обзор l-value и r-value
4. Инициализация ссылок
5. Ссылки в качестве параметров в функциях
6. Ссылки как более легкий способ доступа к данным
7. Ссылки vs. Указатели
8. Заключение

Ссылки

Ссылка — это тип переменной в языке C++, который работает как псевдоним другого объекта или значения. **Язык C++ поддерживает 3 типа ссылок:**

- **Ссылки на неконстантные значения** (обычно их называют просто *«ссылки»* или *«неконстантные ссылки»*), которые мы обсудим на этом уроке.
- **Ссылки на константные значения** (обычно их называют *«константные ссылки»*), которые мы обсудим на следующем уроке.
- В C++11 добавлены **ссылки r-value**, о которых мы поговорим чуть позже.

Ссылка (на неконстантное значение) объявляется с использованием амперсанда (`&`) между типом данных и именем ссылки:

```
int value = 7; // обычная переменная
int &ref = value; // ссылка на переменную value
```

В этом контексте амперсанд не означает «оператор адреса», он означает «ссылка на».

Ссылки в качестве псевдонимов

Ссылки обычно ведут себя идентично значениям, на которые они ссылаются. В этом смысле ссылка работает как псевдоним объекта, на который она ссылается, например:

```
#include <iostream>

int main()
{
    int value = 7; // обычная переменная
    int &ref = value; // ссылка на переменную value

    value = 8; // value теперь 8
    ref = 9; // value теперь 9

    std::cout << value << std::endl; // выведется 9
    ++ref;
    std::cout << value << std::endl; // выведется 10

    return 0;
}
```

Результат выполнения программы:

```
9
10
```

В примере, приведенном выше, объекты `ref` и `value` обрабатываются как одно целое. Использование оператора адреса с ссылкой приведет к возврату адреса значения, на которое ссылается ссылка:

```
std::cout << &value; // выведется 0035FF58
std::cout << &ref; // выведется 0035FF58
```

Краткий обзор l-value и r-value

Мы уже рассматривали, что такое l-value и r-value. l-value — это объект, который имеет определенный адрес памяти (например, переменная `x`) и сохраняется за пределами одного выражения. r-value — это временное значение без определенного адреса памяти и с областью видимости выражения (т.е. сохранется в пределах одного выражения). В качестве r-values могут быть как результаты выражения (например, `2 + 3`), так и **литералы**.

Инициализация ссылок

Ссылки должны быть инициализированы при создании:

```
int value = 7;
int &ref = value; // корректная ссылка: инициализирована переменной value

int &invalidRef; // некорректная ссылка: ссылка должна ссылаться на что-либо
```

В отличие от указателей, которые могут содержать нулевое значение, ссылки нулевыми быть не могут.

Ссылки на неконстантные значения могут быть инициализированы только неконстантными l-values. Они не могут быть инициализированы константными l-values или r-values:

```
int a = 7;
int &ref1 = a; // ок: a - это неконстантная l-value

const int b = 8;
int &ref2 = b; // не ок: b - это константная l-value

int &ref3 = 4; // не ок: 4 - это r-value
```

Обратите внимание, во втором случае вы не можете инициализировать неконстантную ссылку константным объектом. В противном случае, вы бы могли изменить значение константного объекта через ссылку, что уже является нарушением понятия «константа».

После инициализации изменить объект, на который указывает ссылка — нельзя. Рассмотрим следующий фрагмент кода:

```
int value1 = 7;
int value2 = 8;

int &ref = value1; // ок: ref - теперь псевдоним для value1
ref = value2; // присваивание 8 (значение переменной value2) переменной value1. Здесь НЕ изменяется объект, на который ссылается
```

Обратите внимание, во втором стейтменте (`ref = value2;`) выполняется не то, что вы могли бы ожидать! Вместо переприсваивания `ref` ссылаться на переменную `value2`), значение из `value2` присваивается переменной `value1` (на которое и ссылается `ref`).

Ссылки в качестве параметров в функциях

Ссылки чаще всего используются в качестве **параметров** в функциях. В этом контексте ссылка-параметр работает как псевдоним аргумента, а сам аргумент не копируется при передаче в параметр. Это в свою очередь улучшает производительность, если аргумент слишком большой или затратный для копирования.

Мы говорили о том, что передача аргумента-указателя в функцию позволяет функции при разыменовании этого указателя *напрямую* изменять значение аргумента.

Ссылки работают аналогично. Поскольку ссылка-параметр — это псевдоним аргумента, то функция, использующая ссылку-параметр, может изменять аргумент, переданный ей, также *напрямую*:

```
#include <iostream>

// ref - это ссылка на переданный аргумент, а не копия аргумента
void changeN(int &ref)
{
    ref = 8;
}

int main()
{
    int x = 7;

    std::cout << x << '\n';

    changeN(x); // обратите внимание, этот аргумент не обязательно должен быть ссылкой

    std::cout << x << '\n';
    return 0;
}
```

Результат выполнения программы:

```
7
8
```

Когда аргумент `x` передается в функцию, то параметр функции `ref` становится ссылкой на аргумент `x` . Это позволяет функции изменять значение `x` непосредственно через `ref` ! Обратите внимание, переменная `x` не обязательно должна быть ссылкой.

> ***Совет:*** Передавайте аргументы в функцию через неконстантные ссылки-параметры, если они должны быть изменены функцией в дальнейшем.

Основным недостатком использования неконстантных ссылок в качестве параметров в функциях является то, что аргумент должен быть неконстантным l-value (т.е. константой или литералом он быть не может). Мы поговорим об этом подробнее (и о том, как это обойти) на следующем уроке.

Ссылки как более легкий способ доступа к данным

Второе (гораздо менее используемое) применение ссылок заключается в более легком способе доступа к вложенным данным. Рассмотрим следующую **структуру**:

```
struct Something
{
    int value1;
    float value2;
};

struct Other
{
    Something something;
    int otherValue;
};

Other other;
```

Предположим, что нам нужно работать с полем `value1` структуры `Something` переменной `other` структуры `Other` (звучит сложно, но такое также встречается на практике). Обычно, доступ к этому полю осуществлялся бы через `other.something.value1` . А что, если нам нужно неоднократно получать доступ к этому члену? В этом случае код становится громоздким и беспорядочным. Ссылки же предоставляют более легкий способ доступа к данным:

```
int &ref = other.something.value1;
// ref теперь может использоваться вместо other.something.value1
```

Таким образом, следующие два стейтмента идентичны:

```
other.something.value1 = 7;
ref = 7;
```

Ссылки позволяют сделать ваш код более чистым и понятным.

Ссылки vs. Указатели

Ссылка — это тот же указатель, который неявно разыменовывается при доступе к значению, на которое он указывает («под капотом» ссылки реализованы с помощью указателей). Таким образом, в следующем коде:

```
int value = 7;
int *const ptr = &value;
int &ref = value;
```

`*ptr` и `ref` обрабатываются одинаково. Т.е. это одно и то же:

```
*ptr = 7;
ref = 7;
```

Поскольку ссылки должны быть инициализированы корректными объектами (они не могут быть нулевыми) и не могут быть изменены позже, то они, как правило, безопаснее указателей (так как риск разыменования нулевого указателя отпадает). Однако они немного ограничены в функциональности по сравнению с указателями.

Если определенное задание может быть решено с помощью как ссылок, так и указателей, то лучше использовать ссылки. Указатели следует использовать только в тех ситуациях, когда ссылки являются недостаточно эффективными (например, при **динамическом выделении памяти**).

Заключение

Ссылки позволяют определять псевдонимы для других объектов или значений. Ссылки на неконстантные значения могут быть инициализированы только неконстантными l-values. Они не могут быть переприсвоены после инициализации. Ссылки чаще всего используются в качестве параметров в функциях, когда мы хотим изменить значение аргумента или хотим избежать его затратного копирования.